TP1 — Analyse lexicale du langage MICRO

Le programme est écrit en **C**, mais il analyse un fichier écrit dans le langage **MICRO**. Il regroupe les caractères en unités lexicales : mots réservés, identificateurs, nombres, opérateurs et séparateurs.

Sources et références

L'état initial correspond aux photos `IMG_0727.jpg` et `IMG_0728.jpg`, pages imprimées **27 et 28 de l'exercice**. Elles montrent surtout `scanner()` et supposent certaines définitions et fonctions disponibles ailleurs. L'exercice de la page 28 demande d'ajouter les réels comme `5.2`.

Le document théorique utilisé est [MonCourscompilationILIS (1).pdf](<../../cours/MonCourscompilationILIS (1).pdf>). Les références suivantes sont les **pages du PDF du cours**, distinctes des pages photographiées de l'exercice.

Pages du cours	Notions utilisées
18–19	Transformation du texte en tokens ; suppression des espaces et commentaires
20–21	Analyse syntaxique : structure des instructions et expressions
30	Table des symboles et mots-clés
38	Rôle du scanner, suivi des lignes et transmission des tokens
39–40	Unités lexicales, lexèmes et modèles ; exemples de réels
76–77	Expressions et définitions régulières
79 et 100	Automates finis et implémentation d'un analyseur lexical
93	Génération d'un scanner avec Lex/Flex
107	Erreurs lexicales et récupération

Compiler et exécuter

Ouvrir le dossier TP dans VS Code.

- Compiler : `Ctrl+Shift+B` (tache `TP1 : compiler`).
- Executer : menu **Terminal > Executer la tache**, puis `TP1 : executer l'exemple`.

Le resultat est enregistre dans `TP1/unites_lexicales.txt`. Les taches utilisent GCC installe dans `C:\msys64\ucrt64\bin`.

Depuis le terminal PowerShell de VS Code, dans le dossier TP :

```
gcc -std=c11 -Wall -Wextra -Wpedantic .\TP1\tp1.c -o .\TP1\tp1.exe
.\TP1\tp1.exe .\TP1\exemple.txt .\TP1\unites_lexicales.txt
```

Le programme est un analyseur lexical : il faut lui fournir un fichier texte contenant le langage a analyser. Sans cet argument, il affiche `Utilisation` et se termine avec un code d'erreur. Les mots reserves sont en majuscules : `BEGIN`, `END`, `READ`, `WRITE`.

Les entiers produisent `INTLITERAL`, et les nombres decimaux avec un point (`10.32`, `0.5`, `.5`, `12.`) produisent `REALLITERAL`. Le signe reste un operateur separe : `-2.5` produit `MINUSOP`, puis `REALLITERAL`. La notation scientifique (1e3) n'est pas prise en charge.

Sortie utilisée par le TP2

Le fichier `unites_lexicales.txt` est le fichier intermédiaire transmis au TP2. Pour chaque élément reconnu, le TP1 écrit la ligne d'origine, la catégorie du token et le texte exact trouvé :

```
LIGNE    UNITE           LEXEME
-----    -----           ------
1        BEGIN           BEGIN
2        ID              nombre
2        ASSIGNOP        :=
```

`LIGNE` permet de retrouver la position dans le programme, `UNITE` indique la catégorie reconnue et `LEXEME` contient le texte trouvé. Les deux premières lignes sont des en-têtes lisibles ; le TP2 les ignore avant de lire les tokens.

Le flux complet est donc :

```
programme MICRO --TP1 (analyse lexicale)--> unites_lexicales.txt
                                        --TP2 (analyse syntaxique)--> résultat
```

Exemple depuis la racine du dépôt :

```
gcc -std=c11 -Wall -Wextra -Wpedantic .\TP1\tp1.c -o .\TP1\tp1.exe
gcc -std=c11 -Wall -Wextra -Wpedantic .\TP2\tp2.c -o .\TP2\tp2.exe
.\TP1\tp1.exe .\TP1\exemple.txt .\TP1\unites_lexicales.txt
.\TP2\tp2.exe .\TP1\unites_lexicales.txt
```

Le TP1 répond à « quels tokens sont présents ? ». Le TP2 répond ensuite à « ces tokens sont-ils dans le bon ordre ? ». Cette séparation correspond aux deux premières étapes classiques d'un compilateur.

modification de scanner

Cette section détaille l'extension numérique demandée dans l'exercice. **Source du cours : page 40**, modèle des réels ; **pages 76–77**, description des modèles par expressions régulières. La comparaison de tous les autres ajouts et corrections figure dans la section suivante.

Dans `scanner()`, le bloc qui reconnaissait uniquement les entiers a été modifié pour reconnaître aussi les nombres décimaux.

1. Accepter un chiffre ou un point au début

Cette condition permet de traiter aussi un réel comme `.5`. La variable `is_real` indique si le nombre est réel.

```
if (isdigit((unsigned char)in_char) || in_char == '.') {
    int is_real = in_char == '.';

    buffer_char(in_char);
```

2. Vérifier qu'un chiffre suit un point initial

Un point seul n'est pas un nombre. Si aucun chiffre ne suit, le scanner signale une erreur lexicale et remet le caractère suivant dans le fichier.

```
        if (in_char == '.') {
            c = read_char();
            if (c == EOF || !isdigit((unsigned char)c)) {
                unread_char(c);
                lexical_error(in_char);
                continue;
            }
            buffer_char(c);
        }
```

3. Lire les chiffres

Comme auparavant, cette boucle ajoute les chiffres à `token_buffer`.

```
        for (c = read_char();
             c != EOF && isdigit((unsigned char)c);
             c = read_char())
            buffer_char(c);
```

4. Lire la partie décimale après la partie entière

Si un point suit la partie entière, le nombre devient un réel. Le scanner ajoute le point et les chiffres suivants au tampon. Ainsi, `10.32` est lu entièrement dans `token_buffer`. La condition `!is_real` empêche d'accepter un deuxième point dans le même nombre.

```
        if (!is_real && c == '.') {
            is_real = 1;
            buffer_char(c);

            for (c = read_char();
                 c != EOF && isdigit((unsigned char)c);
                 c = read_char())
                buffer_char(c);
        }
```

5. Retourner le bon token

Le caractère suivant est remis dans le fichier avec `unread_char(c)`. Le scanner retourne ensuite `REALLITERAL` pour un réel ou `INTLITERAL` pour un entier.

```
        unread_char(c);
        return is_real ? REALLITERAL : INTLITERAL;
    }
```

Par exemple, pour `10.32;`, le scanner retourne `REALLITERAL` avec le lexème `10.32`, puis analyse `;` à l'appel suivant.

En dehors de `scanner()`, il fallait aussi ajouter `REALLITERAL` dans l'énumération `token` et dans le `switch` de `token_name()` pour afficher le nom de cette nouvelle unité lexicale.

6. Résultat et frontières de cette extension

Texte	Résultat
`10`	`INTLITERAL("10")`
`10.32`	`REALLITERAL("10.32")`

Texte	Résultat
`.5`	`REALLITERAL(".5")`
`12.`	`REALLITERAL("12.")`
`-2.5`	`MINUSOP("-")`, puis `REALLITERAL("2.5")`
`.`	Erreur lexicale

Après des chiffres, un point peut être suivi d'une partie décimale vide : `12.` est accepté. Après un point initial, au moins un chiffre est obligatoire. Le signe reste un opérateur séparé, comme dans le scanner initial.

La condition `!is_real` empêche un deuxième point dans **le même lexème**. Elle ne rejette pas nécessairement tout le texte `1.2.3` : le scanner peut le découper en deux tokens `1.2` et `.3`. Vérifier si une instruction autorise deux nombres successifs relève de l'analyse syntaxique.

Comprendre token, lexème et modèle

Un **token** est la catégorie reconnue ; un **lexème** est le texte concret lu ; un **modèle** décrit les formes acceptées pour cette catégorie.

Token	Lexème possible	Modèle
`ID`	`resultat_1`	Lettre initiale, puis lettres, chiffres ou underscores
`INTLITERAL`	`10`	Suite non vide de chiffres
`REALLITERAL`	`10.32`	Nombre décimal contenant un point
`READ_TOKEN`	`READ`	Mot réservé exact `READ`
`ASSIGNOP`	`:=`	Deux caractères `:` et `=`

`scanner()` retourne le token, `token_buffer` conserve le lexème et `token_name()` donne un nom lisible à la catégorie.

Source du cours : pages 39–40, « Unités lexicales et lexèmes ».

Comparaison complète avec le code initial

Les fonctions auxiliaires seulement appelées sur les photos ne sont pas forcément absentes du document complet. Nous les avons définies dans notre fichier pour obtenir un programme autonome.

1. Définition des catégories et bibliothèques

État initial : le type `token` et des constantes comme `ID`, `LPAREN` et `INTLITERAL` sont utilisés sans que leur définition soit visible.

Ajout : `typedef enum` définit les catégories et `REALLITERAL` ajoute la catégorie des décimaux. `BEGIN_TOKEN`, `END_TOKEN`, `READ_TOKEN` et `WRITE_TOKEN` représentent les mots réservés ; `SCANEOF` marque la fin.

Nous ajoutons `<string.h>` pour `strcmp()` et `<stdlib.h>` pour `exit()` et les codes de retour. `static` limite les fonctions et variables internes à ce fichier C, sans changer les règles MICRO.

Lien avec le cours : pages 39–40. L'énumération représente les catégories lexicales. Les bibliothèques et `static` sont des détails d'organisation C.

2. Définition et protection du tampon

État initial : `extern char token_buffer[]` suppose un tableau défini ailleurs ; `clear_buffer()` et `buffer_char()` sont appelées.

Ajout : nous définissons le tableau, son indice et ces deux fonctions. `clear_buffer()` initialise une chaîne vide ; `buffer_char()` ajoute un caractère et un terminateur `\0`, nécessaire à `strcmp()` et à l'affichage `%s`.

Une vérification empêche le dépassement du tableau. Les 256 emplacements permettent de conserver 255 caractères plus le terminateur. Si le lexème est trop long, un diagnostic est affiché et le programme s'arrête.

Lien avec le cours : page 40. Le tampon conserve le lexème correspondant au modèle. La limite protège la mémoire ; ce n'est pas une règle du cours.

3. Lecture d'un fichier et fonctions auxiliaires

État initial : `getchar()` lit sur l'entrée standard `stdin`.

Modification : `main()` ouvre le fichier fourni en argument avec `fopen(source_name, "r")`. `read_char()` utilise `fgetc(source_file)` et `unread_char()` utilise `ungetc(c, source_file)`.

Cela permet `tp1.exe exemple.txt`. Les fonctions centralisent aussi le suivi des lignes. Le changement de support ne change pas les modèles reconnus.

Lien avec le cours : page 38. L'entrée du scanner est un flux de caractères du programme source ; sa sortie est une suite d'unités lexicales.

4. Gestion correcte de la fin de fichier

État initial : la photo utilise `eof(stdin)`, qui n'est pas une fonction standard C, et ne montre pas de retour après la boucle principale.

Correction : la boucle teste directement la valeur lue :

```
while ((in_char = read_char()) != EOF) {
    /* Reconnaissance. */
}
clear_buffer();
token_line = current_line;
return SCANEOF;
```

`feof()` existe, mais signale une fin déjà rencontrée par une lecture ; il ne prédit pas la lecture suivante. Notre retour explicite évite d'arriver à la fin de la fonction sans valeur. `unread_char()` ignore `EOF` puisque ce marqueur n'est pas un caractère à remettre dans le flux.

Lien avec le cours : page 100. Les appels au scanner se poursuivent jusqu'à la fin du source ; `SCANEOF` arrête ici la boucle principale.

5. Identificateurs et mots réservés

Règle conservée : une lettre, puis zéro ou plusieurs lettres, chiffres ou underscores. Le scanner lit le mot entier avant de le classer.

Ajout : nous implémentons `check_reserved()` avec `strcmp()` pour reconnaître `BEGIN`, `END`, `READ` et `WRITE`. Sinon, elle retourne `ID`. `READING` est donc un identificateur, et `read` aussi puisque la casse compte.

Lien avec le cours : pages 40 et 76–77, modèle des identificateurs. La **page 30** présente une organisation avec table des symboles ; notre version utilise des comparaisons directes et ne construit pas cette table.

6. Réinitialisation du tampon pour chaque tentative

État initial : `clear_buffer()` est appelé au début de `scanner()`.

Modification : il est appelé dans la boucle, après les espaces et avant la reconnaissance. Après une erreur ou un commentaire, le scanner peut continuer dans le même appel : le prochain lexème doit commencer avec un tampon vide. `token_line` est mémorisé à ce même moment.

Lien avec le cours : pages 38–40. Chaque unité a son propre lexème ; les blancs et commentaires ne doivent pas contaminer le token suivant.

7. Lecture en avance et remise du caractère suivant

Principe conservé : le code initial utilise déjà `ungetc()`. Notre fonction `unread_char()` conserve ce mécanisme en ajoutant le suivi des lignes.

Pour `10.32;`, le scanner lit `;` pour savoir que le nombre est terminé, puis le remet dans le flux. Le premier appel retourne le réel ; le suivant retourne `SEMICOLON`. Sans cette remise, le séparateur serait perdu.

Les variables de lecture restent de type `int`, comme sur les photos, pour représenter les caractères et le marqueur `EOF`.

Lien avec le cours : pages 79 et 100. Cette lecture permet de déterminer la frontière du lexème lors de la reconnaissance.

8. Organisation des symboles et conservation de leur texte

État initial : les symboles sont reconnus par une succession de `else if`.

Modification : un `switch` regroupe ces tests, sans ajouter d'opérateurs. `buffer_char(in_char)` conserve le lexème des symboles. Pour `:=`, le `=` est aussi ajouté au tampon après sa vérification.

La règle initiale est conservée : `:` doit être suivi de `=`. Sinon, le caractère suivant est remis dans le flux et `:` provoque un diagnostic.

Lien avec le cours : pages 18 et 39. Les opérateurs et séparateurs sont des unités lexicales ; certains lexèmes comportent plusieurs caractères.

9. Arrêt des commentaires à la fin du fichier

Règle conservée : `-` donne `MINUSOP`, `--` commence un commentaire.

Correction : la boucle initiale attendait seulement \n. Un commentaire en fin de fichier sans retour à la ligne pouvait provoquer une boucle infinie. Nous ajoutons le test de fin de fichier :

```
while (in_char != '\n' && in_char != EOF);
```

Les commentaires sont ignorés et ne produisent pas de token.

Lien avec le cours : pages 18–19 et 38. Le scanner élimine les espaces et commentaires avant de transmettre les unités utiles.

10. Numéros de ligne et diagnostics

Ajout : `current_line` suit la lecture ; `token_line` conserve la ligne de début du lexème. `read_char()` incrémente le compteur sur \n ; `unread_char()` le décrémente si ce caractère est remis, pour ne pas compter deux fois la même ligne.

Nous définissons `lexical_error()`, appelée mais non montrée sur les photos. Elle affiche le fichier, la ligne et le caractère incorrect, ou sa valeur hexadécimale s'il n'est pas imprimable, puis incrémente `error_count`. Le scanner abandonne le caractère incorrect et continue.

À la fin, le programme retourne un échec si des erreurs ont été détectées. Un lexème trop long entraîne, lui, un arrêt immédiat dans `buffer_char()`.

Lien avec le cours : page 38, suivi des lignes ; **page 107**, erreurs lexicales et récupération en ignorant les caractères problématiques.

11. Tests de caractères sécurisés en C

Modification : les appels aux fonctions de <ctype.h> utilisent des conversions comme `isdigit((unsigned char)c)`. Ces fonctions attendent EOF ou une valeur représentable comme `unsigned char`.

Les boucles vérifient aussi `c != EOF` avant la classification des caractères. Ces précautions rendent les appels valides en C ; elles n'ajoutent aucune nouvelle catégorie au langage.

Lien avec le cours : page 38, tâche de classification du scanner. Les conversions sont une précaution d'implémentation C, pas une règle théorique.

12. Programme principal et sortie lisible

Ajout : `main()` vérifie les arguments, ouvre les fichiers, traite les échecs d'ouverture, appelle le scanner jusqu'à SCANEOF et ferme les fichiers.

```
while ((current_token = scanner()) != SCANEOF) {
    fprintf(output_file, "%-8u %-16s %s\n",
            token_line, token_name(current_token), token_buffer);
}
```

`token_name()` donne le nom de la catégorie. Le résultat contient la ligne, le token et le lexème. SCANEOF termine la boucle sans être écrit dans la table.

Les tâches VS Code, l'exemple et ce README facilitent la compilation et l'étude ; ils ne modifient pas les modèles lexicaux.

Lien avec le cours : pages 38 et 100. Dans un compilateur, les tokens sont transmis à l'analyseur syntaxique. Ici, ils sont enregistrés pour observer la première étape.

Expressions régulières et automates

Modèles de notre scanner

Pour les caractères ASCII, les modèles peuvent être écrits ainsi :

```
ID          : [A-Za-z][A-Za-z0-9_]*
INTLITERAL  : [0-9]+
REALLITERAL : ([0-9]+\.[0-9]*|\.[0-9]+)
```

* signifie zéro ou plusieurs occurrences, + une ou plusieurs, | une alternative, et `\.` un point littéral. Dans le C, les classes de caractères sont reconnues avec les fonctions de `<ctype.h>`.

Source du cours : pages 76–77. Une expression régulière décrit toutes les formes possibles des lexèmes d'une catégorie.

Le bloc numérique comme un automate

État conceptuel	Caractère	Action
Début	Chiffre	Commencer la partie entière
Début	Point	Exiger un chiffre pour former un réel
Partie entière	Chiffre	Continuer l'entier
Partie entière	Point	Passer à la forme décimale
Partie décimale	Chiffre	Continuer la partie décimale
Nombre reconnu	Autre caractère ou EOF	Terminer ; remettre le caractère s'il existe

Les boucles correspondent aux répétitions, les conditions aux changements d'état et `is_real` mémorise le passage à la forme décimale.

Source du cours : pages 79 et 100. Un langage régulier est reconnu par un automate fini ; un scanner peut simuler cette reconnaissance. La **page 93** présente Lex/Flex, qui génère ce programme depuis des spécifications régulières. Notre scanner est écrit à la main.

Exemple complet

```
BEGIN
READ(nombre);
resultat := nombre + 10.32 - 2;
WRITE(resultat);
END
```

Pour la troisième ligne, les tokens sont :

Ligne	Token	Lexème
3	`ID`	`resultat`
3	`ASSIGNOP`	`:=`
3	`ID`	`nombre`

Ligne	Token	Lexème
3	PLUSOP	+
3	REALLITERAL	10.32
3	MINUSOP	-
3	INTLITERAL	2
3	SEMICOLON	;

Les espaces sont ignorés ; le scanner ne calcule pas l'expression.

Source du cours : pages 18–19 et 38–40. C'est le passage d'une suite de caractères à une suite d'unités lexicales.

Limites actuelles et distinction avec les autres phases

Notre extension se limite aux décimaux avec point. 1e3 est découpé en `INTLITERAL("1")` et `ID("e3")`, sans erreur lexicale automatique.

+ et - comme opérateurs séparés.

Leur ordre et la structure des expressions relèvent de l'analyse syntaxique présentée aux pages 20–21, non implémentée ici.

sans construire la table présentée aux pages 19, 30 et 38.

ni les types et n'exécute pas les instructions MICRO.

d'identificateurs Unicode ; chaque lexème est limité à 255 caractères.

- **Exposants :** la page 40 présente des réels comme `0.5e3` et `-4e-1`.
- **Signes :** le cours donne aussi des modèles signés ; notre scanner garde
- **Syntaxe :** reconnaître les tokens ne garantit pas une instruction valide.
- **Table des symboles :** nous conservons les identificateurs comme lexèmes,
- **Sémantique et exécution :** le programme ne vérifie ni les déclarations
- **Alphabet et taille :** la lecture se fait par octets, sans reconnaissance

La réponse centrale à l'exercice est l'ajout du modèle décimal et du token REALLITERAL. Les autres corrections et ajouts rendent cette reconnaissance utilisable dans un programme C autonome, avec une sortie lisible et des diagnostics.